Monitoring & Alerting

How PhysioGuidance is observed in production — in-app infra dashboard, GCP uptime checks, log-based alerts, Sentry error monitoring, health endpoints, long-term log retention, and email alert routing.

PREPARED FOR
PhysioGuidance Client Team

DATE
2026-08-17

ALERT CHANNELS WIRED
nelc.platform@gmail.com (Amna) + ghulam@physioguidance.org (transition)

DEPLOYED COMMIT
d396b040

PHYSIOGUIDANCE

Riyadh · Saudi Arabia
me-central2 · GCP Cloud Run

Table of contents

1. Executive summary
2. Observability stack — bird's-eye view
3. Admin Monitoring dashboard (in-app)
4. Cloud Run service health
5. Cloud Run Job execution visibility
6. Cloud SQL and Redis metrics
7. GCS bucket size tracking
8. Uptime checks
9. Log-based alert policies
10. Sentry — error monitoring
11. Sentry webhook fan-out to admins
12. Health endpoints
13. Long-term log retention
14. Alert routing and recipients
15. Component status matrix

---

1. Executive summary

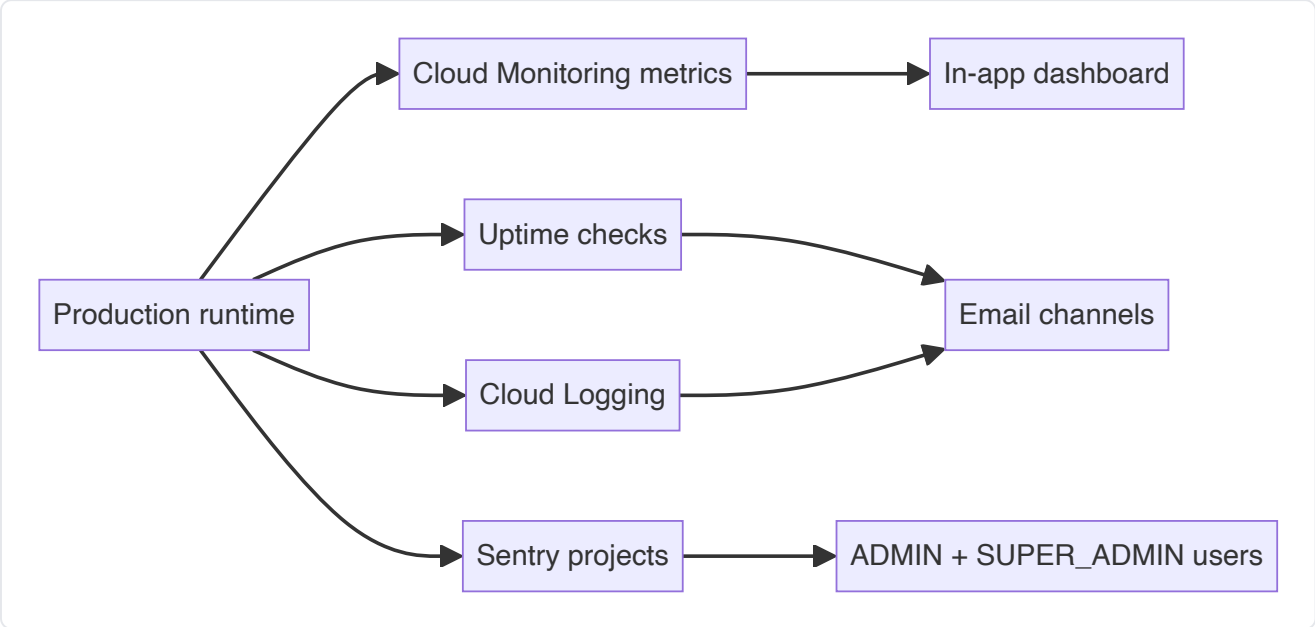
Owner-facing observability lives in four coordinated surfaces:

1. **In-app Monitoring dashboard** at `/dashboard/admin/monitoring` — real-time infra pane for the owner without leaving the platform. Cards for Services, Uptime, Jobs, Cloud SQL, Redis, and Buckets, auto-refreshed every 30 seconds.
2. **Cloud Monitoring alert policies** — six policies covering site-down, backend 5xx spikes, job execution failures, failed-login bursts, suspicious 403 bursts, and malware detections.
3. **Sentry** — application error monitoring for backend (`@sentry/nestjs`) and frontend (`@sentry/nextjs`). A Sentry-signed webhook fans every issue-level alert out to all ADMIN and SUPER_ADMIN users through an in-app notification + email.
4. **Cloud Logging** with 400-day retention on the `_Default` and `_Required` buckets, feeding the log-based metrics that drive alerts and audit evidence.

All GCP alert channels notify `nelc.platform@gmail.com` (Amna's project inbox). Ghulam's channel remains attached during handover and is removed with the one-command script in `alert-recipients-status.md` .

---

2. Observability stack — bird's-eye view



3. Admin Monitoring dashboard (in-app)

Route: `/dashboard/admin/monitoring`, guarded by `JwtAuthGuard` + `RolesGuard` for `ADMIN` and `SUPER_ADMIN`. Throttling skipped so a busy owner can refresh freely; a 60-second in-process cache in `MonitoringService` collapses concurrent calls so admins do not fan out into Cloud Monitoring API cost.

Six cards, auto-refreshing every 30 seconds via `Promise.allSettled` so a single degraded card leaves the others intact:

Card	Data source	Content
Services	Backend `fetch()` against each service's health URL	api / web / cms up-down + latency; worker + malware-scanner probed via GCP-side metrics
Uptime	`monitoring.googleapis.com/uptime_check/check_passed` metric	Aggregated per configured Uptime Check ID
Jobs	`run.googleapis.com/job/completed_execution_count`	Succeeded / failed / last-run for each of 25 Cloud Run Jobs over the last 24h
Cloud SQL	`cloudsql.googleapis.com/database/*`	CPU %, memory %, active connections, disk bytes
Redis	`redis.googleapis.com/stats/*` and `clients/*`	Memory %, memory bytes, connected clients, ops/sec
Buckets	`storage.googleapis.com/storage/total_bytes`	Per-bucket total bytes across all 6 project buckets

Backend endpoints (all under `/admin/monitoring/*`):

- `GET /admin/monitoring/services-health`
- `GET /admin/monitoring/uptime`
- `GET /admin/monitoring/jobs-stats?hours=24`
- `GET /admin/monitoring/cloudsql`
- `GET /admin/monitoring/redis`
- `GET /admin/monitoring/buckets`

Graceful degradation: if the api-runtime service account is missing `roles/monitoring.viewer` on any call, the response includes `iamGrantPending: true` and the widget renders a friendly placeholder instead of a 500. This lets the UI ship ahead of the IAM grant.

Each card carries an "Open in GCP" link that deep-links to the equivalent Cloud Console view for detailed drill-down.

---

4. Cloud Run service health

- **Data source:** in-band `fetch()` from the api service to each other service's health URL, plus the GCP-side Uptime checks in section 8.
- **Services probed:** `physioguidance-api`, `physioguidance-web`, `physioguidance-cms`, `physioguidance-api-worker` (metric-only, internal), `physioguidance-malware-scanner` (Eventarc-triggered, metric-only).

- **In-app view:** Monitoring dashboard → Services grid — up / down / unknown with HTTP status and latency.
- **GCP-native view:** Cloud Console → Cloud Run → services list.

---

5. Cloud Run Job execution visibility

- **Data source:** `run.googleapis.com/job/completed_execution_count` aligned by `job_name` with the `result` label ("succeeded" / "failed").
- **In-app view:** Monitoring dashboard → Jobs table — succeeded, failed, last-execution timestamp per job over a caller-controlled window (default 24h, capped at 168h).
- **GCP-native view:** Cloud Console → Cloud Run → Jobs → per-job execution history.
- **Alert:** Cloud Run Job execution failure is one of the six alert policies (section 9 + `alert-recipients-status.md`).

Covers all 25 background jobs including `physio-job-payments-tamara-retry`, `physio-job-video-transcode`, `physio-job-xapi-flush`, `physio-job-support-sla-breach`, and `physio-job-zoom-sync`.

---

6. Cloud SQL and Redis metrics

6.1 Cloud SQL

Pulled from Cloud Monitoring over a 10-minute window per refresh:

- `cloudsql.googleapis.com/database/cpu/utilization`
- `cloudsql.googleapis.com/database/memory/utilization`
- `cloudsql.googleapis.com/database/postgresql/num_backends`
- `cloudsql.googleapis.com/database/disk/bytes_used`

6.2 Redis (Memorystore)

Pulled from Cloud Monitoring over the same 10-minute window:

- `redis.googleapis.com/stats/memory/usage_ratio`
- `redis.googleapis.com/stats/memory/usage`
- `redis.googleapis.com/clients/connected`
- `redis.googleapis.com/commands/processed_count`

Both cards apply the same `iamGrantPending` guard as the other widgets.

---

7. GCS bucket size tracking

- **Data source:** `storage.googleapis.com/storage/total_bytes` on a 48-hour lookback window (GCS updates this metric once per day — the wider window ensures a value is available on every refresh).
- **In-app view:** Monitoring dashboard → Buckets card — per-bucket size for all 6 project buckets.
- **Buckets tracked:** `physioguidance-backups`, `physioguidance-cms`, `physioguidance-private`, `physioguidance-public`, `physioguidance-quarantine`, `physioguidance-trainers`.
- Useful for spotting runaway growth in user uploads, backup accumulation, or quarantine bloat.

---

8. Uptime checks

Two Cloud Monitoring uptime checks poll the public entrypoints from six global probe regions every 60 seconds:

Check	URL	Frequency
`physioguidance-backend`	`https://api.physioguidance.org/api/health/ready`	60s
`physioguidance-frontend`	`https://physioguidance.org/`	60s

Both feed the "Site or API down" alert policy in section 9. Failure notifications flow to Amna and Ghulam via the email channels in section 14.

---

9. Log-based alert policies

Six Cloud Monitoring policies are wired to both notification channels. Four have JSON definitions in the repo under `infra/monitoring/`. The remaining two — site-down and job-failure — are configured directly on the project.

Policy	Trigger	Source
Site or API down	Uptime check failing for ≥2 minutes on either the backend or frontend probe	Uptime metric `monitoring.googleapis.com/uptime_check/check_passed`
Backend 5xx spike	>20 5xx responses on `physioguidance-api` in a 5-minute window	Log-based metric `backend_5xx_count` (`infra/monitoring/alert-5xx-spike.json`)
Cloud Run Job execution failed	Any of the 25 jobs completes with `result=failed`	`run.googleapis.com/job/completed_execution_count`
Failed login burst	>30 401s on `/api/auth/login` in 5 minutes	Log-based metric `auth_login_401_count` (`infra/monitoring/alert-failed-login-burst.json`)
Suspicious 403 burst	>50 403s on the backend in 5 minutes	Log-based metric `backend_403_count` (`infra/monitoring/alert-suspicious-403.json`)
Malware detected	Any `malware_scan.infected` structured-log entry from the scanner service	`infra/monitoring/alert-malware-detected.json` (log-based match)

Each policy embeds a Markdown runbook in its `documentation.content` — visible in the Cloud Console alert view and in the notification email. Auto-close is set to 30 minutes for the metric-threshold alerts and 7 days for the malware alert (so a single detection stays open until acknowledged).

Re-applying a deleted policy is a one-liner using the JSON template:

```
gcloud alpha monitoring policies create \
  --policy-from-file=infra/monitoring/alert-5xx-spike.json
```

Full recovery playbook — including how to recreate the log-based metrics the alerts depend on — lives in `infra/monitoring/README.md`.

---

10. Sentry — error monitoring

10.1 Backend

- SDK: `@sentry/nestjs` with `@sentry/profiling-node`.
- Init lives in `backends/src/instrument.ts` and is imported **before** `AppModule` so global instrumentation catches every unhandled exception + auto-instruments HTTP and DB spans.
- Sampling: `SENTRY_TRACES_SAMPLE_RATE` and `SENTRY_PROFILES_SAMPLE_RATE` default to 0.1.

- Known-noise filter: connection-timeout errors from Cloud SQL public-IP SSL churn are dropped via `ignoreErrors`.
- Missing `SENTRY_DSN` makes the SDK a no-op so the service still boots without the secret set.

10.2 Frontend

- SDK: `@sentry/nextjs` — Node runtime init in `frontend/src/sentry.server.config.ts`, edge runtime in `sentry.edge.config.ts`, browser init via `instrumentation-client.ts`.
- Enabled only when `NEXT_PUBLIC_APP_ENV` is `production` or `staging`.
- `sendDefaultPii: false` and a `scrubEvent` `beforeSend` hook strip request-body and header PII before events leave the browser.
- Source maps upload at build time (`SENTRY_AUTH_TOKEN` in the Docker build context) so production stack traces are readable.

10.3 Admin Sentry panel

Route: Settings → System → Error Monitoring, backed by `/admin/error-monitoring/*`:

- `GET overview` — 24h / 7d / 30d event + user counts, plus a dashboard link.
- `GET top` — unresolved issues sorted by frequency for the period.
- `GET trend` — bucketed timeline of recent issues.
- `GET issue/:id` — full Sentry issue detail.
- `POST issue/:id/resolve` — mark resolved (Sentry is the source of truth so no local audit row is duplicated).

Sentry API host is configurable via `SENTRY_API_HOST` (defaults to `de.sentry.io`, matching the EU data-residency plan on the current org).

---

11. Sentry webhook fan-out to admins

Any Sentry alert rule can POST to `https://api.physioguidance.org/api/webhooks/sentry/alert`. The receiver in `error-monitoring-webhook.controller.ts`:

1. Verifies the `sentry-hook-signature` header against `SENTRY_WEBHOOK_SECRET` using an HMAC-SHA256 timing-safe compare. Missing secret in production rejects the request; missing outside production accepts with a warning log.
2. Invalidates the admin-dashboard cache so the next refresh reflects the just-arrived event.
3. Filters on `sentry-hook-resource` = `issue` / `error` / `event_alert`.
4. Calls `ErrorMonitoringNotificationsService.fanOut(...)`:
   - Loads every user carrying the `ADMIN` or `SUPER_ADMIN` role.
   - Creates a bell-badge in-app notification for each (respecting each admin's own push preferences via `UserNotificationsService.createBulk`).

- Sends each admin a localized email summary via `EmailService.sendSentryAlert(...)` with the issue title, culprit, event count, affected users, and a deep-link back to the admin Error Monitoring panel.

Result: an admin does not need a Sentry seat to hear about a production error — the alert reaches them inside PhysioGuidance and in their inbox.

---

12. Health endpoints

Three endpoints under `/api/health`, all marked `@Public()`:

Endpoint	Purpose	Behavior
`GET /api/health`	Baseline liveness	Returns `{status, timestamp, environment, uptime}` — always 200 if the process responds
`GET /api/health/live`	Cloud Run liveness probe	Returns `{status, timestamp}` — always 200 if the process responds
`GET /api/health/ready`	Full readiness probe	Pings Postgres (`SELECT 1`) and Redis (`PING`) with a 2-second timeout each

`/api/health/ready` returns HTTP 200 with `status: "ok"` when both dependencies respond. If Postgres fails, it throws `ServiceUnavailableException` (HTTP 503) with the failing check surfaced in the body. Redis is reported inline as `{ok: false, error: ...}` without failing the request, so a transient Memorystore hiccup does not knock the backend out of rotation for stateless traffic.

Sample response for the healthy path:

```
{
  "status": "ok",
  "timestamp": "2026-08-17T08:22:11.252Z",
  "checks": {
    "database": { "ok": true, "latencyMs": 7 },
    "redis": { "ok": true, "latencyMs": 17 }
  }
}
```

The backend uptime check in section 8 hits `/api/health/ready`, so a Postgres outage flips the uptime probe → the "Site or API down" alert fires within two minutes.

---

13. Long-term log retention

Cloud Logging retention is set to 400 days on both the `_Default` and `_Required` log buckets:

Bucket	Retention	Content
`_Default`	400 days	Cloud Run request logs, application logs, log-based metric source
`_Required`	400 days	GCP admin actions, IAM changes, security-relevant infrastructure events

This exceeds the PDPL Article 18 evidence-retention window used elsewhere in the audit posture and gives investigators a full year plus a rollover buffer to reconstruct events. Verify at any time with `gcloud logging buckets list --location=global`.

---

14. Alert routing and recipients

Every GCP alert policy notifies both channels below. See `alert-recipients-status.md` for the full ownership audit + the one-command handover-day script that detaches Ghulam's channel from every policy in a single loop.

Channel	Email	Purpose
Amna (PM)	`nelc.platform@gmail.com`	Primary — client-owned inbox
Ghulam (dev)	`ghulam@physioguidance.org`	Secondary during handover; removed on cutover day

Optional defense-in-depth documented in `alert-recipients-status.md`: swap Amna's Gmail for a `alerts@physioguidance.org` group alias so on-call rotation changes stay outside GCP.

Sentry notifications reach the same audience through a different path — the webhook fan-out in section 11 delivers to every ADMIN / SUPER_ADMIN in the platform (in-app + email), independent of the GCP channel list.

---

15. Component status matrix

Amna's four-category vocabulary. Only items Amna asked about + items shipped.

Item	Status	Notes
In-app Monitoring dashboard	Implemented	Six cards live at `/dashboard/admin/monitoring`, 30s auto-refresh, 60s server cache, graceful IAM-pending state
Cloud Run service in-band health probe	Implemented	Backend fans out `fetch()` to api / web / cms; worker + malware-scanner probed via GCP metrics
Cloud Run Job execution visibility	Implemented	25 jobs surface succeeded / failed / last-run in the dashboard and via `run.googleapis.com/job/completed_execution_count`
Cloud SQL live metrics	Implemented	CPU, memory, connections, disk from `cloudsql.googleapis.com/database/*`
Redis live metrics	Implemented	Memory usage, connected clients, ops from `redis.googleapis.com/stats/*`
GCS bucket size tracking	Implemented	Per-bucket totals for all 6 buckets on the 48h GCS lookback
Uptime checks	Implemented	`physioguidance-backend` on `/api/health/ready` and `physioguidance-frontend` on `/` every 60s from 6 regions
Site-or-API-down alert	Implemented	Fires on ≥2 min of failed uptime probes; wired to both channels
Backend 5xx spike alert	Implemented	`alert-5xx-spike.json` — >20 5xx per 5 min
Cloud Run Job failure alert	Implemented	Fires on any job execution with `result=failed`
Failed login burst alert	Implemented	`alert-failed-login-burst.json` — >30 × 401 on `/api/auth/login` per 5 min
Suspicious 403 burst alert	Implemented	`alert-suspicious-403.json` — >50 × 403 per 5 min
Malware detection alert	Implemented	`alert-malware-detected.json` — fires on every `malware_scan.infected` structured-log entry
Backend Sentry SDK	Implemented	`@sentry/nestjs` initialised before `AppModule`; DSN via `SENTRY_DSN` secret
Frontend Sentry SDK	Implemented	`@sentry/nextjs` for browser + node + edge runtimes; source maps uploaded at build time
Admin Error Monitoring panel	Implemented	Overview + top issues + trend + issue detail + resolve, all under `/admin/error-monitoring/*`
Sentry alert webhook fan-out	Implemented	HMAC-verified webhook fans issues to every ADMIN / SUPER_ADMIN via in-app notification + email
`/api/health/ready` real dependency check	Implemented	Live Postgres `SELECT 1` + Redis `PING`, HTTP 503 on database failure
Long-term log retention	Implemented	400 days on `_Default` and `_Required`, verifiable via `gcloud logging buckets list`
Alert channels routed to Amna	Implemented	`nelc.platform@gmail.com` attached to all 6 policies; Ghulam retained as secondary until cutover

Item	Status	Notes
Handover-day Ghulam-removal script	Implemented	Provided in `alert-recipients-status.md`

16. Where to look next

- **How the Monitoring dashboard is built** → `gcp-deployment.md` and the `MonitoringModule` in `backends/src/modules/monitoring/`
- **How to add or change an alert policy** → edit the JSON under `infra/monitoring/`, then `gcloud alpha monitoring policies create --policy-from-file=...`
- **How to add another notification channel** → GCP Console → Monitoring → Alerting → Notification Channels
- **Full alert-recipient audit + handover-cutover script** → `alert-recipients-status.md`
- **Cost impact of Sentry + Monitoring API calls** → `cost-tally.md`
- **What a Redis / DB restart looks like operationally** → `redis-persistence-risk.md`, `regional-failure-playbook.md`